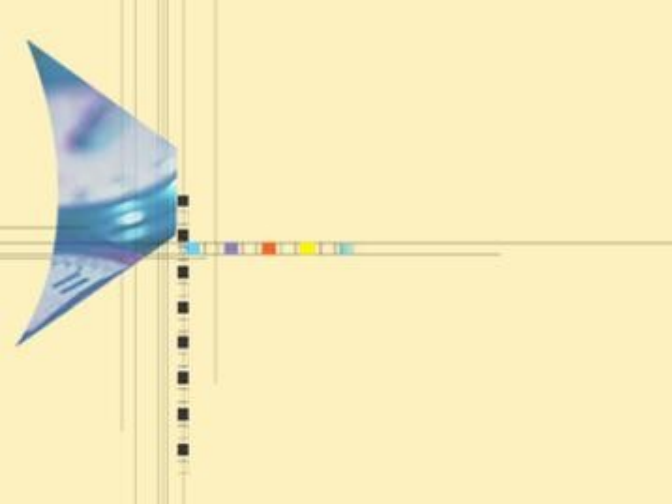




贪心算法

——一种求解最优化问题的有效算法

武汉大学 刘斌





5 贪心算法

- 5.1 问题引入
- 5.2 贪心准则 / 贪心度量
 - 5.2.1 分数背包
 - 5.2.2 活动安排
- 5.3 贪心算法一般框架及特点分析
- 5.4 贪心解为最优解的正确性证明
- 5.5 贪心策略和动态规划策略的异同点
- 5.6 贪心策略在其他课程算法中的体现

5.1 问题引入（选自算法导论）

- 给你一个整数数组 `prices`，其中 `prices[i]` 表示某支股票第 i 天的价格。
- 在每一天，你可以决定是否购买和 / 或出售股票。你在任何时候最只能持有一股股票，且只能交易一次。
- 返回你能获得的最大利润。



5.1 解法（选自算法导论）

- 暴力解法
- 分治法（最大连续子数组的和）
- 动态规划法
- 贪心解法
 - 第 i 天获得的最大利润—— $\text{price}[i] - \min(\text{price}[0..i-1])$
 - 全局最大利润 $\max(\text{price}[i] - \min(\text{price}[0..i-1]))$



5.1 解法（选自算法导论）

- 贪心解的优点
 - 好理解、简单
 - 得到最优解?
- 本题目只有一个因数，股票的价格，当有多个因数需要考虑时，贪心策略（准则）如何选择？



5.2.1 贪心准则——分数背包问题

? 问题描述：

- 给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的项目, 并设背包容量为 C , 要找到非负实数 $x_1, x_2, \dots, x_n$ 使和

$$\sum_{i=1}^n x_i v_i$$

满足下面约束条件, 取得最大值。

$$\sum_{i=1}^n x_i s_i \leq C$$

5.2.1 贪心准则——分数背包问题

- $n=3, C=20$

C=20	物品 1	物品 2	物品 3
价值	25	24	15
重量	18	15	10

- 可行方案：

方案	(x_1, x_2, x_3)	重量 ($\sum s_i x_i$)	价值 ($\sum v_i x_i$)
1	方案	$9+5+2.5=16.5$	$12.5+8+3.75=24.25$
2	方案	$18+2=20$	$25+3.2=28.2$
3	方案	20	31
4	方案	20	31.5

5.2.1 贪心准则——分数背包问题选择策略 1

- 从剩下的物品中选择最大价值的物品装入背包，得到：
- 方案(2) : $(1, \frac{2}{15}, 0)$ 20 28.2 (非最优)
- 原因：虽然价值很大，但是容量的消耗太快

C=20	物品 1	物品 2	物品 3
价值	25	24	15
重量	18	15	10

5.2.1 贪心准则——分数背包问题选择策略

- 从剩下的物品中选择最小尺寸的物品装入背包。

• 方案(3): $(0, \frac{2}{3}, 1)$ 20 31 (非最优)

- 原因: 虽然容量损耗较少, 但是价值增长速度太慢

C=20	物品 1	物品 2	物品 3
价值	25	24	15
重量	18	15	10

5.2.1 贪心准则——分数背包问题选择策略

3

- 从剩下的物品中选择最大 v_i/s_i 比率 的物品装入背包。

• 方案 (4): $(0, 1, \frac{1}{2})$ 20 31.5 (最优)

C=20	物品 1	物品 2	物品 3
价值	25	24	15
重量	18	15	10



5.2.1 贪心准则——分数背包问题最优策略

- Step 1:
 - 每项计算 $y_i = v_i / s_i$, 即该项值和大小的比
- Step 2:
 - 再按比值的降序来排序, 从第一项开始装背包, 然后是第二项, 依次类推, 尽可能地多放, 直至装满背包。

算法 Greedy-KNAPSACK

输入：按照 $v(i)/s(i)$ 比率降序排列的 n 个物品的容量数组 $s[1..n]$ 和物品价值数组 $v[1..n]$ ；背包的总容量 C ，物品的数量 n

输出：最优贪心算法结果 $x[1..n]$

```
for  $i \leftarrow 0$  to  $n$     { 初始化  $x[i]$  }
```

```
     $x[i] \leftarrow 0$ 
```

```
end for
```

```
 $cu \leftarrow C$ 
```

```
for  $i \leftarrow 0$  to  $n$     { 物品按次序装入背包 }
```

```
    if  $s[i] > cu$  then
```

```
        exit
```

```
    end if
```

```
     $x[i] \leftarrow 1$ 
```

```
     $cu \leftarrow cu - s[i]$     { 剩余背包容量 }
```

```
end for
```

```
if  $i \leq n$  then  $x[i] \leftarrow cu/s[i]$  end if { 最后剩余背包容量中装入物品  $i$  的比率 }
```

```
return  $x$ 
```

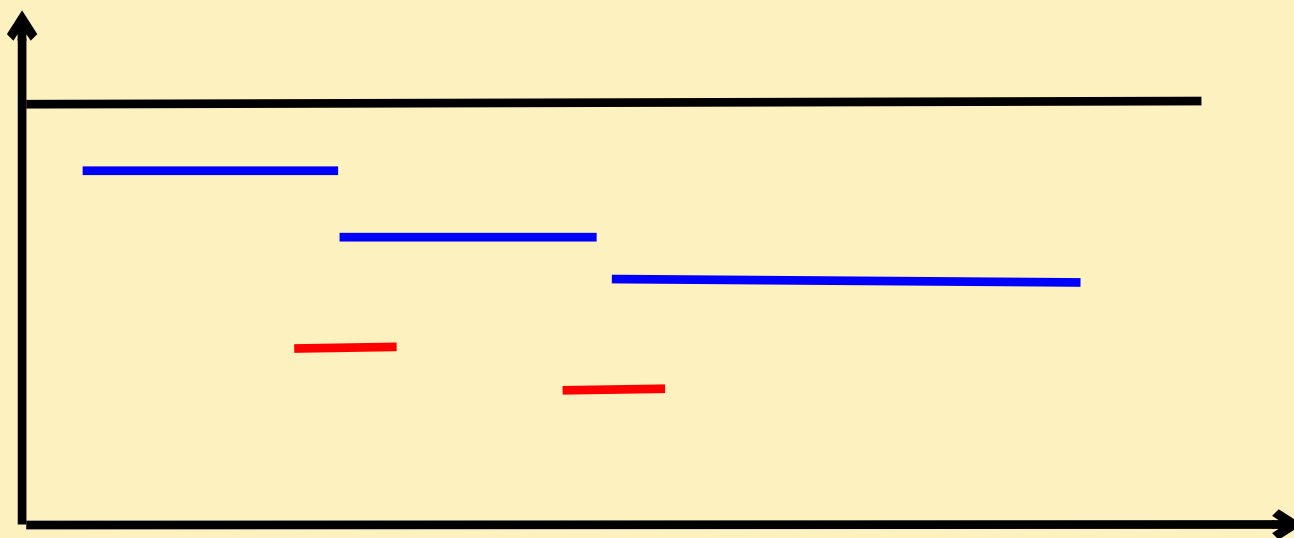
5.2.2 活动安排问题

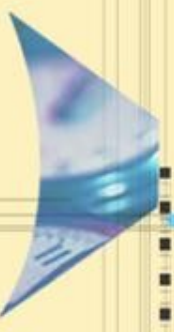
【问题描述】 假设有一个需要使用某一资源（教室）的 n 个活动所组成的集合 S ， $S = \{1, \dots, n\}$ 。该资源任何时刻只能被一个活动所占用，活动 i 有一个开始时间 b_i 和结束时间 e_i （ $b_i < e_i$ ），其执行时间为 $e_i - b_i$ ，假设最早活动执行时间为 0。一旦某个活动开始执行，中间不能被打断，直到其执行完毕。若活动 i 和活动 j 有 $b_i \geq e_j$ 或 $b_j \geq e_i$ ，则称这两个活动兼容。

【问题要求】 设计算法求一种最优活动安排方案，使得所有安排的活动个数最多。

5.2.2 活动安排问题

	活动 1	活动 2	活动 3	活动 4	活动 5	活动 6
开始时间	0	1	5	11	4	10
结束时间	20	5	11	18	6	12

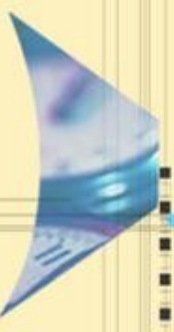




分数背包 vs 活动安排

- 物品：重量、价值两个参数
- 活动：开始时间、结束时间两个参数
- 背包：有重量限制，价值最大
- 安排：活动越多越好（可以认为背包重量无穷大，每一个活动的价值是 1）





5.2.2 贪心准则——活动安排

- 策略一：选择具有最早开始时间，而且不与已安排的活动冲突的活动。
- 策略二：选择具有最短使用时间，而且不与已安排的活动冲突的活动。
- 策略三：选择具有最早结束时间，而且不与已安排的活动冲突的活动。

5.2.2 贪心准则——活动安排问题选择策略 1

- 开始最早的活动优先，目标是想尽早结束活动，让出教室。
- **这个显然不行，因为最早的活动可能很长，影响我们进行后面的活动。**
- 安排 $[0,20)$ 的这个活动之后，其他活动无法安排。

	活动 1	活动 2	活动 3	活动 4	活动 5	活动 6
开始时间	0	1	5	11	4	10
结束时间	20	5	11	18	6	12

5.2.2 贪心准则——活动安排问题选择策略 2

- 短活动优先，目标尽量少占资源，空出更多的时间教室。
- 安排活动 $[4,6)$ ， $[10,12)$ ，但是短活动正好在两个活动之间，形成“加塞”，可见这个贪心策略也是不行的。

	活动 1	活动 2	活动 3	活动 4	活动 5	活动 6
开始时间	0	1	5	11	4	10
结束时间	20	5	11	18	6	12

5.2.2 贪心准则——活动安排问题选择策略 3

- 选择具有最早结束时间，希望空出更多的时间留给其他活动。
- 安排活动 $[1,5)$ ， $[5,11)$ ， $[11,18)$ ，最多的不冲突的活动数目是 3。

	活动 1	活动 2	活动 3	活动 4	活动 5	活动 6
开始时间	0	1	5	11	4	10
结束时间	20	5	11	18	6	12

例如，对于下表的 $n=11$ 个活动（已按结束时间递增排序）

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动集合的过程：

活动 1	✓
活动 2	×
活动 3	×
活动 4	✓
活动 5	×
活动 6	×
活动 7	×
活动 8	✓
活动 9	×
活动 10	×
活动 11	✓

最大兼容活动集合：

活动 1 活动 4 活动 8 活动 11

1 4 8

└──────────────────┘

求解结果

// 问题表示

```
struct Action          // 活动的类型声明
{
    int b;              // 活动起始时间
    int e;              // 活动结束时间
    bool operator<(const Action &s) const // 重载 < 关系函数
    {
        return e<=s.e; // 用于按活动结束时间递增排序
    }
};
int n=11;
Action A[]={ {0}, {1,4}, {3,5}, {0,6}, {5,7}, {3,8}, {5,9}, {6,10},
{8,11},
{8,12}, {2,13}, {12,15}}; // 下标 0 不用
```

// 求解结果表示

```
bool flag[MAX]; // 标记选择的活动
int Count=0;     // 选取的兼容活动个数
```

```
void solve()
{
    memset(flag,0,sizeof(flag));
    sort(A+1,A+n+1);
    int preend=0;
    for (int i=1;i<=n;i++)
    {
        if (A[i].b>=preend)
        {
            flag[i]=true;
            preend=A[i].e;
        }
    }
}
```

// 求解最大兼容活动子集
// 初始化为 false
// A[1..n] 按活动结束时间递增排序
// 前一个兼容活动的结束时间
// 扫描所有活动
// 找到一个兼容活动
// 选择 A[i] 活动
// 更新 preend 值

5.2.2 活动选择问题——算法分析

【算法分析】 算法的主要时间花费在排序上，排序时间为 $O(n\log_2 n)$ ，所以整个算法的时间复杂度为 $O(n\log_2 n)$ 。



5.2.2 贪心准则总结

- 贪心算法产生全局最优解的最重要因素是**选择策略**。**选择策略**不能简单的从给的因数中加以选择，有时需要对相关的因数进行进一步处理，得到贪心选择的策略
- 因为每一步的工作很少且基于少量信息，所得算法特别有效。
- **贪心算法适合求解最大值 / 最小值问题。**

5.3 贪心算法的框架

Algorithm GREEDY(A,n)

 solution $\leftarrow \emptyset$

 for $i \leftarrow 1$ to n do

$x \leftarrow \text{SELECT}(A)$

 if FEASIBLE(solution,x)

 then solution $\leftarrow \text{UNION}(\text{solution},x)$

 end if

 end for

 return (solution)



5.3 贪心算法——货郎担问题不能得到最优解

- 货郎担问题也叫旅行商问题，即 TSP 问题（Traveling Salesman Problem），是数学领域中著名问题之一。
- **问题描述：**某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员选择出发的城市及旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。
- 相当于求距离最短的哈密尔顿回路，需求从各城市出发的最短回路，再对 n 条回路求极小值。
- 回路最短——贪心选择距离短的路线

5.3 货郎担问题

- 例：假设 5 个城市费用矩阵为：

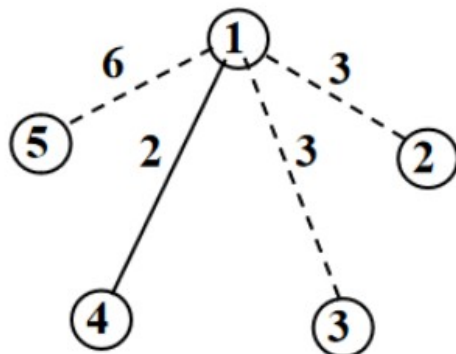
	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

- 从城市 1 出发 :1->4->3->5->2->1 , 总距离 14
- 从城市 2 出发 :2->5->4->1 —>3->2 , 总距离 17
- 其它城市同理, 再在 5 条回路中选择最小距离。

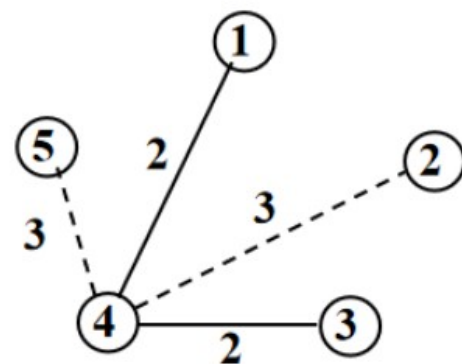
5.3 货郎担问题——最近邻点贪心策略

$$C = \begin{pmatrix} \infty & 3 & 3 & 2 & 6 \\ 3 & \infty & 7 & 3 & 2 \\ 3 & 7 & \infty & 2 & 5 \\ 2 & 3 & 2 & \infty & 3 \\ 6 & 2 & 5 & 3 & \infty \end{pmatrix}$$

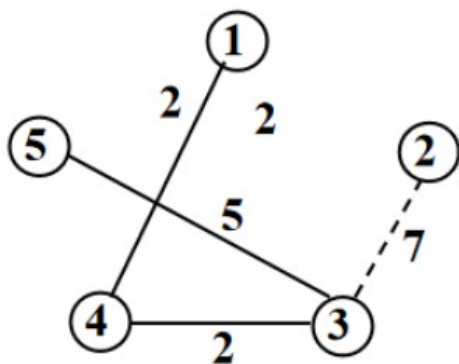
(a) 5城市的代价矩阵



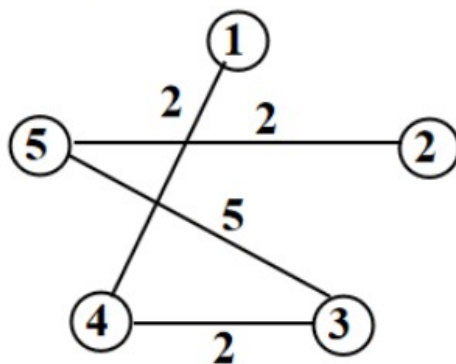
(b) 城市1→城市4



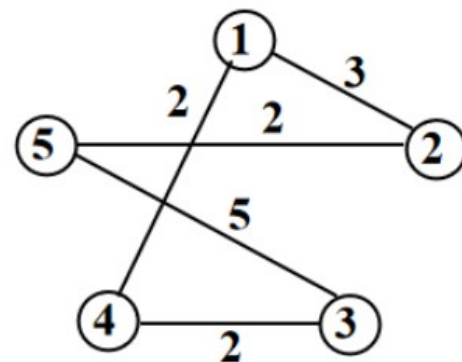
(c) 城市4→城市3



(d) 城市3→城市5



(e) 城市5→城市2



(f) 城市2→城市1

最近邻点贪心策略求解TSP问题的过程，求出的最短路径为1-4-3-5-2-1，长度为14

5.3 货郎担问题

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

- 从城市 1 出发 :1->4->3->5->2->1 , 总距离 14
- 从城市 2 出发 :2->5->4->1->3->2 , 总距离 17
- 其它城市同理, 再在 5 条回路中选择最小距离。

5.3 贪心算法特点

- 对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
- 但是，从许多可用贪心算法求解的问题中我们看到这类问题一般具有 2 个重要的性质：**贪心选择性质**和**最优子结构性质**。



5.3 贪心算法——贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
- 动态规划算法通常以自底向上的方式解各子问题，而贪心算法则通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

5.3 贪心算法——最优子结构性

- 当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性**。问题的最优子结构性是该问题可用动态规划算法或贪心算法求解的关键特征。
- 贪心算法和动态规划算法都要求问题具有最优子结构性，这是 2 类算法的一个共同点。



5.4 贪心算法最优解的证明

- 贪心算法最难的部分从不在于问题的求解，而在于正确性的证明，常用的证明方法有归纳法和交换论证法。
- 交换论证法：从最优解出发，在保证最优性不变的前提下，从一个最优解进行逐步替换，从而得到贪心策略的解。
- 归纳法：对算法进行步数归纳或问题规模归纳
 - 算法步骤进行归纳
 - 问题规模归纳

5.4 活动选择问题——算法证明

步骤	内容	例子
1	确定贪心策略	活动按结束时间递增排序，下一个任务的开始时间大于前一个任务的结束时间。
2	贪心选择性质，即贪心选择必包含于某一个最优解中。	证明最先结束的任务（编号为1）必在一个最优解中。
3	最优子结构	若 X 是活动安排问题 A 的最优解， $X = X' \cup \{1\}$ ，则 X' 是 $A' = \{i \in A : e_i \geq b_1\}$ 的活动安排问题的最优解。



5.4 活动选择问题——算法证明

- (1) 对所有的活动按照结束时间升序排序。
- (2) 证明总存在一个以活动 1 开始的最优解。

如果第一个选中的活动为 k ($k \neq 1$)，第二个活动为 j ，可以构造另一个最优解 Y ， Y 中的活动是兼容的， Y 与 X 的活动数相同。

那么用活动 1 取代活动 k 得到 Y' ，因为 $e_1 \leq e_k < b_j$ ，所以 Y' 中的活动是兼容的，即 Y' 也是最优的，这就说明总存在一个以活动 1 开始的最优解。

5.4 活动选择问题

(3) 当做出了对活动 1 的贪心选择后，原问题就变成了在活动 2、…、n 中找与**活动 1 兼容**的那些活动的子问题。亦即，如果 X 为原问题的一个最优解，则 $X' = X - \{1\}$ 也是活动选择问题 $A' = \{i \in A \mid b_i \geq e_1\}$ 的一个最优解。

反证法：如果能找到一个 A' 的含有比 X' 更多活动的解 Y' ，则将活动 1 加入 Y' 后就得到 A 的一个包含比 X 更多活动的解 Y ，这就与 X 是最优解的假设相矛盾。

因此，在每一次贪心选择后，留下的是一个与原问题具有相同形式的最优化问题，即最优子结构性质。

5.4 算法证明——平均等待时间最短

- n 个作业，每一个作业的运行时间为 $t_i (1 \leq i \leq n)$
- 作业的等待时间是排在前面的所有作业及该作业本身执行时间之和。
- 平均等待时间为所有作业的等待时间的平均值：

$$\bar{w} = \frac{\sum_{i=1}^n w_i}{n}$$

贪心策略：对所有的作业按照执行时间从小到大排序。
执行时间最短的任务最先执行。



贪心选择性质

- 贪心选择：执行时间最短的任务最先执行包含在最优解中。
- 反证法。构造一个最优解，假定最先执行的任务是 t_k ，不是 t_1 。设执行时间最短的任务最先执行不包含在任何一个最优解中。

$$T = n * t_1 + (n - 1) * t_2 + \dots + (n - k) * t_k + \dots + t_n$$

$$T' = n * t_k + (n - 1) * t_2 + \dots + (n - k) * t_1 + \dots + t_n$$

显然 $T < T'$ ，即交换 t_k 和 t_1 ，总的等待时间更少，所以假设不成立。最优解的第一个作业必须是 t_1

最优子结构性质

- 当做出了对作业 1 的贪心选择后，原问题就变成了其余 $n-1$ 个作业的排序问题。
- 剩余作业也需要求出最少等待时间的执行序列，与原问题性质相同，因此也必须最短作业优先。子问题原来的问题性质相同。
- 最优子结构得到证明。



最小耗费生成树——Kruskal 算法

- 设 $G = (V, E)$ 是一个具有含权边的连通无向图。 G 的一棵**生成树** (V, T) 是 G 的作为树的子图。如给 G 加权并且 T 的各边的权的和为最小值，那么 (V, T) 就称为**最小耗费生成树**或简称为**最小生成树**。
- 假设 G 为连通的，怎么找到最小生成树(MST)?

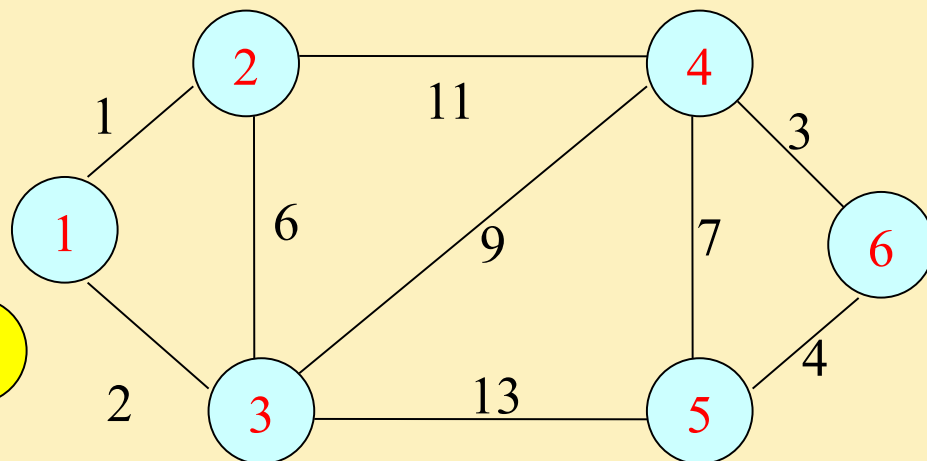
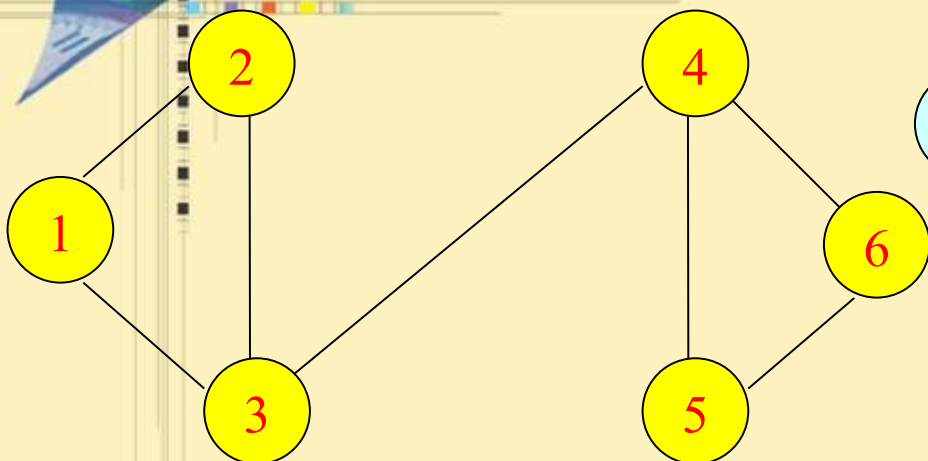


最小耗费生成树——Kruskal 算法

- 工作原理：维护由许多生成树组成的一个森林，这些生成树逐步合并直到最终森林只由一棵树组成，这棵树就是最小耗费生成树。
- Step 1: 首先对所有边的权以非降序排列。
- Step 2: 接着从由图的顶点组成而不包含边的森林 (V, T) 开始，重复下面的这一步，直到 (V, T) 被变换成一棵树：
 - 设 (V, T) 是到现在为止构建的森林， $e \in E - T$ 为当前考虑的边，如果把 e 加到 T 中不生成一个回路，那么将 e 加入进 T ，否则丢弃。
 - 这个处理在恰好加完 $n-1$ 条边后结束。



例子



(1,2)	(1,3)	(4,6)	(5,6)	(2,3)	(4,5)	(3,4)	(2,4)	(3,5)
1	2	3	4	6	7	9	11	13

! 构成回路，这条边被丢弃。

Success!

算法 5.1.1 KRUSKAL

输入：包含 n 个顶点的含权连通无向图 $G=(V,E)$ 。

输出：由 G 生成的最小耗费生成树 T 组成的边的集合。

1. 按非降序权重将 E 中的边排序

2.for 每个顶点 $v \in V$

3. MAKESET{ v }

4.end for

5. $T=\{\}$

6.**while** $|T| < n-1$

7. 令 (x,y) 为 E 中的下一条边。

8. **if** $\text{FIND}(x) \neq \text{FIND}(y)$ **then** { 检查 x , y 是否在同一颗树中 }

9. 将 (x,y) 加入 T

10 **UNION**(x,y)

11 **end if**

12. **end while**



最小耗费生成树——算法步骤归纳证明

- T 是最小生成树的边集合，对 T 的大小进行归纳法证明。
- 假设最终的最小生成树的边集合为 T_{min}
- 初始（第5步 $T=\{\}$ ）命题平凡为真。
- 第七步之前得到的最小边集合为 T ，此时加入的边为 $e(x,y)$ ，那么 $T' = T \cup \{e\}$ ，必须证明 $T' \subseteq T_{min}$
 - T_{min} 恰好包含 e ，无须证明。
 - 假设 T_{min} 不包含 e ，即算法没有选择 e ，而是选择了比 e 的权值大的边生成最小生成树。
 - 此时将 e 加入到 T_{min} 中，可以得到一条回路，删除该回路中权值最小的边，得到的还是一颗树，权值更小。
 - 所以假设不成立， T_{min} 必定包含 e 。

装载问题——问题规模归纳法证明

- 最优装载问题：
 - n 个集装箱 $1, 2, \dots, n$ 装上轮船，集装箱 i 的重量为 w_i ，轮船载重量限制为 c ，无体积限制。如何使装上船的集装箱最多。（假设每个集装箱重量小于 c ）
- 贪心策略：将集装箱按照从轻到重排序，轻者先装
- 正确性证明：对规模归纳
 - 设箱子标号按照从轻到重记为 $1, 2, \dots, n$
 - $n = 1$ 贪心选择显然得到最优解
 - 假设对规模 n 的输入得到最优解，证明对规模 $n+1$ 的输入也得到最优解

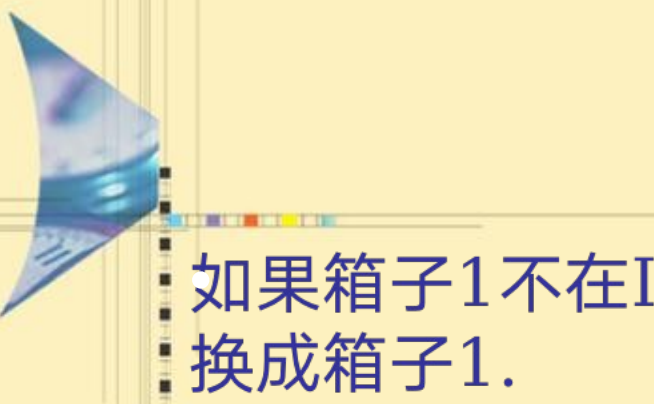
- (1) 当 $n=1$ 时, 因为只有一个箱子, 所以显然成立
- (2) 假设 $n=N$ 时, 贪心算法可以得到最优解 (此处 N 是问题的规模)

(3) 当 $n=N+1$ 时, 令 $C' = C - w_1$,

$$N' = \{2, 3, \dots, N + 1\}$$

对于 N' 采用贪心策略得到的解为 I' , 需要证明 $I = I' \cup \{1\}$ 是 $N+1$ 时的解。

假设 I 不是 $N+1$ 时的最优解, 设最优解是 I^* , 那么 $|I^*| > |I|$ 。



如果箱子1不在 I^* 中可以将最优解的重量最小的箱子替换成箱子1.

那么 $I^* - \{1\}$ 是 N' 的最优解。

$$|I^* - \{1\}| > |I' - 1\{1\}| = I'$$

则与 I' 是 N' 的最优解相矛盾，所以假设不成立。

所以 I 是算法的最优解，即当 $n=N+1$ 时，用贪心策略找到的解是最优解。



5.5 贪心策略和动态规划策略的异同点

- 与动态规划算法比较：
 - 贪心算法通常用来于求解最优化问题，即量的最大化或最小化。
 - 然而，贪心算法不像动态规划算法，它通常包含一个用以寻找局部最优解的迭代过程。在某些实例中，这些局部最优解转变成了全局最优解，而在另外一些情况下，则无法找到最优解。

5.6 贪心算法在其他课程算法中的体现

- 数据库系统实现——左深树连接算法
- 机器学习——梯度下降

